



Fullstack Web Development

Session 9 | Week 11/Day 2 | 120 min

Laravel - Part06 - The END

Instructor: Ms. Liev Nova



090667393



Outline

- Authentication in Laravel
- How Laravel Authentication Works
- Session-Based Authentication in Laravel
- Token-Based Authentication in Laravel
- Implement Authentication and API with Laravel Sanctum

Authentication in Laravel

- **What is Authentication?**
 - Authentication is the process of verifying a user's identity.
 - Example: A user enters an email and password.
 - Laravel checks:
 - Does the email exist?
 - Is the password correct?
 - Should this user be allowed to enter the system?
 - If the credentials are correct, the user is authenticated.

Authentication in Laravel

- **What is Authentication?**
 - In a Student Management System, authentication protects pages such as:
 - Dashboard
 - Student list
 - Student create form
 - Student edit form
 - Gender management

Authentication vs Authorization

- Authentication and authorization are related, but they are not the same.
 - Authentication answers:
 - Who are you?
 - Authorization answers:
 - What are you allowed to do?
- Example:
 - Login checks authentication.
 - Checking whether a user can delete a student is authorization.

Authentication vs Authorization

- Authentication and authorization are related, but they are not the same.
 - Authentication answers:
 - Who are you?
 - Authorization answers:
 - What are you allowed to do?
- Example:
 - Login checks authentication.
 - Checking whether a user can delete a student is authorization.

What Laravel Authentication Provides

- Laravel provides authentication tools for common web application needs.
 - Laravel can help with:
 - User registration
 - Login
 - Logout
 - Password hashing
 - Session management
 - Route protection
 - Email verification
 - Password reset
 - API token authentication
- Laravel gives secure building blocks so developers do not need to write everything from zero.

Authentication Options in Laravel

- Laravel supports different ways to build authentication.
 - Common options:
 - Starter Kits
 - Fortify
 - Custom authentication
 - Sanctum
 - Passport
- Starter kits are useful for quickly adding login and registration.
- Custom authentication is useful for learning how authentication works internally.
- Sanctum is useful for API tokens, SPAs, and mobile apps.
- Passport is useful when an application needs full OAuth2 support.

Laravel Starter Kits

- Laravel starter kits provide ready-made authentication scaffolding.
 - They commonly include:
 - Register page
 - Login page
 - Logout
 - Password reset
 - Profile features
 - Why use starter kits?
 - Faster setup
 - Good default structure
 - Good learning reference
 - Even if students later write custom authentication, starter kits are useful to study how Laravel structures auth routes, controllers, and views.

Core Components of Laravel Authentication

- Laravel authentication has four important parts:
 - Guards
 - Providers
 - Middleware
 - User model
- Simple explanation:
 - Guard defines how users are authenticated.
 - Provider defines where users are loaded from.
 - Middleware protects routes.
 - User model represents authenticated users.
- These settings are configured mainly in: [config/auth.php](#)

Guards

- A guard defines how Laravel authenticates users for each request.
- Common guard: **web**
- The **web** guard uses:
 - Sessions
 - Cookies
 - Browser requests
- Example use:
 - When a user logs in through a normal web form, Laravel usually uses the **web** guard.
- For API authentication, Laravel Sanctum commonly uses: **auth:sanctum**

Providers

- A provider defines how users are retrieved from storage.
 - Default provider: `users`
 - The default users provider usually loads users from: `App\Models\User`
 - The `User model` connects to the `users database table`.
 - The guard checks the request. The provider loads the user.

Middleware

- Middleware can protect routes.
 - Common authentication middleware: **auth**
 - Example:

```
Route::middleware('auth')->group(function () {  
    Route::get('/dashboard', [DashboardController::class, 'index'])  
        ->name('dashboard');  
});
```

- This means: Only logged-in users can access **/dashboard**.

Guest Middleware

- The `guest` middleware is used for visitors who are not logged in.

- Example:

```
Route::middleware('guest')->group(function () {  
    Route::get('/login', [AuthController::class, 'showLogin'])  
        ->name('login');  
  
    Route::get('/register', [AuthController::class, 'showRegister'])  
        ->name('register');  
});
```

- Why?

- Logged-in users should not need to visit the login or register page again.

User Model

- The User model represents users who can log in.
 - File: `app/Models/User.php`
 - Common fields:
 - `name`
 - `email`
 - `password`
 - `remember_token`
 - Important: Passwords must be hashed before saving.
 - Example:

```
Hash::make($request->password)
```
 - Never store plain text passwords.

Session-Based Authentication

- Session-based authentication is Laravel's default web authentication style.
- It is used for traditional web apps.
- How it works:
 - User logs in through a form.
 - Laravel verifies the email and password.
 - Laravel stores login information in the server session.
 - Browser stores a session cookie.
 - Future requests send the cookie automatically.

Session-Based Authentication

- Session-based authentication is Laravel's default web authentication style.
- It is used for traditional web apps.
- Use session authentication for:
 - Admin dashboard
 - Student Management System web pages
 - Blade-based web applications

Session Authentication Workflow

- Workflow:
 1. User submits login form.
 2. Laravel checks the credentials.
 3. If correct, Laravel creates a session.
 4. Laravel sends a session cookie to the browser.
 5. Browser sends the cookie on future requests.
 6. Laravel uses the session to identify the user.
- Important:
 - The user does not need to send email and password on every request.
- The browser session remembers the logged-in user.

Login with Auth::attempt

- Laravel can check login credentials using `Auth::attempt()`.

- Example:

```
use Illuminate\Support\Facades\Auth;

$credentials = $request->validate([
    'email' => ['required', 'email'],
    'password' => ['required'],
]);

if (Auth::attempt($credentials)) {
    $request->session()->regenerate();

    return redirect()->intended(route('dashboard'));
}

return back()->withErrors([
    'email' => 'Invalid email or password.',
]);
```

- `Auth::attempt()` returns true if the credentials are correct.

Why Regenerate Session?

- After login, Laravel should regenerate the session.
 - Example:

```
$request->session()->regenerate();
```
 - Why?
 - It improves security.
 - It helps prevent session fixation attacks.
 - It gives the logged-in user a fresh session ID.
- Always regenerate the session after successful login.

Logout

- Logout removes the authenticated user from the session.

- Example:

```
Auth::logout();  
  
$request->session()->invalidate();  
$request->session()->regenerateToken();  
  
return redirect()->route('login');
```

- - `Auth::logout()` logs out the user.
- - `invalidate()` clears the session.
- - `regenerateToken()` creates a new CSRF token.

Protect SMS Routes

- In the Student Management System, dashboard pages should be protected.

- Example:

```
Route::middleware('auth')->group(function () {  
    Route::get('/dashboard', [DashboardController::class, 'index'])  
        ->name('dashboard');  
  
    Route::resource('students', StudentController::class);  
    Route::resource('genders', GenderController::class);  
});
```

- Result:

- Only logged-in users can access dashboard, students, and genders.

Access the Logged-In User

- Laravel provides helper methods to access the current user.

- Using `auth()`:

```
auth()->user()
```

- Using request:

```
$request->user()
```

- Using Auth facade:

```
Auth::user()
```

- Example in Blade:

```
Welcome, {{ auth()->user()->name }}
```

Token-Based Authentication

- Token-based authentication is commonly used for APIs.
 - Instead of browser sessions, the client sends a token.
 - Common clients:
 - Mobile app
 - JavaScript frontend
 - Third-party system
 - Postman
 - The token is usually sent in the request header: **Authorization: Bearer token_here**

Token Authentication Workflow

- Workflow:
 1. User sends email and password to the API.
 2. Server verifies credentials.
 3. Server creates an API token.
 4. Client stores the token.
 5. Client sends the token with future API requests.
 6. Server validates the token and identifies the user.
- Important:
 - API clients usually do not use normal browser sessions.

Session-Based vs Token-Based Authentication

- Comparison:
 - Session-Based
 - Used for web apps
 - Uses cookies
 - Server stores session data
 - Good for Blade dashboards
 - Token-Based
 - Used for APIs
 - Uses Authorization header
 - Client sends token on each request
 - Good for mobile apps and external clients

What is an API?

- API means Application Programming Interface.
- In Laravel, an API allows another application to communicate with your system.
 - Example:
 - A mobile app wants to get student data from the Student Management System.
 - The mobile app sends a request: `GET /api/students`
- Laravel returns data as `JSON`.

Web Route vs API Route

- Web routes usually return Blade views.

- Example:

```
return view('pages.students.index', compact('students'));
```

- API routes usually return JSON.

- Example:

```
return response()->json([  
    'data' => $students,  
]);
```

What is JSON?

- JSON is a common data format used by APIs.

- Example:

```
{  
  "id": 1,  
  "name": "Sok Dara",  
  "email": "sok.dara@example.com"  
}
```

- Why use JSON?

- Easy for frontend apps to read
- Easy for mobile apps to read
- Smaller than HTML
- Common standard for APIs

What is Laravel Sanctum?

- Laravel Sanctum is a lightweight authentication package for:
 - API tokens
 - Single-page applications
 - Mobile applications
- Sanctum can:
 - Create API tokens
 - Store hashed tokens in the database
 - Check Bearer tokens
 - Protect API routes
 - Revoke tokens

Install API Support

- Laravel 12 can install API support using:

```
php artisan install:api
```

- Then run:

```
php artisan migrate
```

- This prepares the project for API routes and Sanctum authentication.
- **Important:**
 - If `routes/api.php` does not exist before, this command helps set up API routing.

Add HasApiTokens to User Model

- To create API tokens, the User model needs the `HasApiTokens` trait.

- File: `app/Models/User.php`

- Example:

```
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;
}
```

- `HasApiTokens` gives the User model token methods such as `createToken()`.

Bearer Token

- After login, the API returns a token.
- The client must send the token in future requests.
- Header: **Authorization: Bearer your_token_here**
- Example:

```
GET /api/students
Authorization: Bearer 1|abc123...
```
- The token proves that the **API request** belongs to an authenticated user.



THANK YOU

For your attention !

